

P2.9b resource spike — record

Project: network-visualiser
Run: 22 September 2026, Streamlit Community Cloud
Status: passed; P2.9b builds in-process with the runtime lock unchanged

Question

Can Community Cloud install the runtime lock exactly as committed, and run the heavy half of the in-app build (sentence-transformers embeddings and a 3-D UMAP fit) on 442 rows, with room to spare? The answer decides P2.9b’s design, so it was measured before any of P2.9b was written.

Method

A throwaway app (`spike_app.py`) on a throwaway branch (`spike/resources`), never merged into `main`, deployed with Python 3.11 and no secrets. It used only the tracked synthetic export, in which every row is invented; no API key and no Claude calls. It read the container’s memory limit from the system rather than assuming one, and timed each step with memory measured after it.

The decision rule was set before the run:

- **Pass:** the install succeeds from `requirements.txt` unchanged, peak memory is at most 60% of the container limit, and embedding plus both UMAP fits take at most 60 seconds. P2.9b then builds in-process with the lock as it is.
- **Otherwise:** a second spike with the CPU-only torch wheel; if that also failed, P2.9b’s design would change.

Results

Measure	Result	Rule	
Install from <code>requirements.txt</code>	Succeeded; 97 packages, about 36 s from “processing dependencies” to done	must succeed	pass
Container memory limit	3,072 MB	—	
Peak resident memory	1,332 MB (43% of the limit)	at most 60%	pass
Embedding (0.5 s) + first UMAP fit (10.4 s) + second fit (0.6 s)	11.5 s	at most 60 s	pass
Reload after the first run	Instant (result cached)	—	

Other measurements:

Python	3.11.16
CPUs visible	16
torch	2.13.0+cu130: a CUDA build, with no GPU visible
Disk: torch / nvidia, cuda and triton packages	1,044 MB / 3,443 MB
Import torch, sentence-transformers, umap	36.0 s, 1,137 MB resident
Load all-MiniLM-L6-v2 (first download)	3.8 s
First page load, script start to results	51.3 s

Decision

P2.9b builds in-process, with the runtime lock unchanged. The CPU-only torch spike is not needed.

What the numbers mean for P2.9b’s design

- **Import cost dominates, not the build.** Importing the heavy libraries takes 36 s and 1.1 GB; embedding and

projecting 442 rows takes 11.5 s. So the heavy imports must stay out of the landing page: the synthetic demo loads without them, and they load once, when the first real build is requested. The loaded model is shared across sessions (`st.cache_resource`), as the spike did.

- **The first UMAP fit per process pays about 10 s of one-off compilation.** Later fits are under a second.
- **Memory headroom is about 1.7 GB above one loaded model.** The build steps added about 150 MB on top of the loaded model at 442 rows, part of it UMAP's one-off compilation; how many concurrent builds to allow is a P2.9b limit to set, not a blocker.
- **The CUDA packages are dead weight here** (3.4 GB on disk, no GPU). They do not break anything and the rule passed, so this stays as it is. Switching to the CPU-only wheel is a possible later optimisation for cold-start time, to be measured if cold start becomes a problem.

Found by the spike

- **Community Cloud overrode the lock for one package.** It replaced `pyarrow==25.0.0` with 24.0.0 at deploy, citing a known segfault (apache/arrow#50471). The lock now pins `pyarrow==24.0.0`, so the locked environment matches the one that runs (D-52).
- **Harmless log noise, for P2.9:** Streamlit's file watcher logs repeated "Examining the path of torch.classes" errors once torch is imported, and UMAP warns that `random_state` forces single-threaded fitting (deliberate: it keeps the layout reproducible). The file-watcher noise can be silenced at P2.9.

The spike app and branch were deleted after this record was written.